

E-Commerce Customer Intelligence Platform: NLP Sentiment Analysis, Churn Prediction & Customer Segmentation

Executive Summary

Modern e-commerce businesses generate large volumes of transactional, behavioral, and customer feedback data, yet many organizations struggle to transform this information into timely decisions. This project developed an end-to-end E-Commerce Intelligence Platform that converts fragmented customer data into predictive and actionable business intelligence.

The platform integrates Data Engineering, SQL Analytics, Machine Learning, Customer Segmentation, NLP, Deep Learning, and Business Intelligence to move beyond traditional reporting toward proactive decision-making.

The project began with building a robust data pipeline to clean, validate, and transform raw customer data. Behavioral features such as Recency, Frequency, Monetary Value (RFM) and customer engagement metrics were engineered to understand customer value and purchasing behavior.

For customer churn prediction, multiple machine learning models, including Logistic Regression, Random Forest, and XGBoost, were developed. To address severe class imbalance, SMOTE with Tomek Links was applied, and models were optimized using recall and F1-score to prioritize identifying customers at risk of leaving. The final model improved churn detection performance from 35% to 97%, enabling earlier retention actions.

Customer segmentation was performed using K-Means and DBSCAN clustering, creating actionable customer personas such as Premium, Loyal, Risky, and Inactive customers. These segments support personalized marketing campaigns, retention strategies, and customer lifetime value optimization.

To understand customer voice, an NLP pipeline was developed to analyze product reviews using preprocessing techniques, TF-IDF, deep learning models, and BERT Transformer fine-tuning with Hugging Face. The sentiment analysis model achieved 91% accuracy, allowing businesses to identify customer satisfaction trends and emerging issues.

A SQL analytics layer was also developed using SQLite to provide business users with accessible insights into revenue trends, customer behavior, and product performance without requiring machine learning expertise. The final platform combines predictive analytics and customer intelligence into a unified decision-support system, helping organizations transition from reactive reporting to proactive strategies for customer retention, targeted marketing, improved customer experience, and sustainable revenue growth.

Chapter 1: The Business Problem

In most e-commerce environments, customer data exists in fragmented silos. Customer demographics reside in one system, purchase history in another, transaction records in a third, and product reviews in yet another platform. While data is abundant, it is not connected, and this disconnection leads to critical business limitations.

Customers likely to churn are not identified early enough to take preventive action. Marketing campaigns remain generic instead of personalized, wasting budget on ineffective outreach. Customer reviews and feedback remain underutilized, missing opportunities to improve products and services. Executives rely solely on historical dashboards that describe what has already happened rather than predicting what will happen next. Revenue opportunities are consistently lost due to poor behavioral understanding and delayed response times.

The result is a costly imbalance: companies continuously invest heavily in acquiring new customers while failing to retain existing high-value ones. Customer acquisition costs continue to rise, yet retention strategies remain reactive and poorly targeted. This project solves that problem by building a unified Customer Intelligence Platform capable of predicting behavior, analyzing sentiment, and guiding business actions before revenue is lost.

Chapter 2: Project Objectives and Scope

The platform was designed around five core objectives that address the fundamental gaps in traditional e-commerce analytics.

Objective 1: Predict customer churn before customers leave. The system needed to identify at-risk customers with sufficient lead time to enable proactive retention interventions. This required supervised machine learning models optimized for recall to minimize false negatives.

Objective 2: Understand customer opinions using NLP. Customer reviews contain rich, unstructured data about satisfaction, dissatisfaction, and emerging issues. The platform needed to automatically extract sentiment and detect early warning signals from textual feedback.

Objective 3: Segment customers for personalized marketing. One-size-fits-all marketing strategies are inefficient and often ineffective. The platform needed to identify distinct behavioral customer personas to enable targeted, personalized engagement strategies.

Objective 4: Provide interactive dashboards for business users. Insights needed to be accessible to non-technical stakeholders without requiring machine learning expertise. This required a self-service analytics layer with intuitive query capabilities.

Objective 5: Build an end-to-end production-ready ML pipeline. The system needed to be scalable, maintainable, and robust enough to handle real-world data challenges including missing values, class imbalance, and noisy text.

Chapter 3: Data Sources and Engineering Pipeline

The platform integrates both structured and unstructured data to form a **4 datasets**. Structured data sources included customer demographic information, sales transaction records, product catalog details, payment method data, and revenue records. Unstructured data sources consisted of product reviews, customer feedback text, numerical ratings, and sentiment labels.

Feature	Description
CustomerID	Unique identifier for each customer.
Churn	Target variable indicating whether the customer has churned (Yes/No).
Tenure	Number of months the customer has remained with the company.
PreferredLoginDevice	Customer's most frequently used login device.
CityTier	Classification of the customer's city based on development level.
WarehouseToHome	Distance from the warehouse to the customer's home.
PreferredPaymentMode	Customer's preferred payment method.
Gender	Customer gender.
HourSpendOnApp	Average hours spent using the mobile app or website.
NumberOfDeviceRegistered	Total registered devices associated with the customer.
PreferredOrderCat	Customer's preferred product category.
SatisfactionScore	Customer satisfaction rating.
MaritalStatus	Marital status of the customer.
NumberOfAddress	Number of delivery addresses registered.
Complain	Indicates whether the customer submitted a complaint.
OrderAmountHikeFromLastYear	Percentage increase in annual order value compared with previous year.
CouponUsed	Number of coupons used by the customer.
OrderCount	Total number of orders placed.
DaySinceLastOrder	Days since the customer's last purchase.
CashbackAmount	Average cashback amount received.

Data Engineering Pipeline

High-quality insights depend on high-quality data, and this principle guided the design of the comprehensive data engineering pipeline. The first stage involved rigorous data cleaning to ensure

data integrity. Missing values were systematically treated through imputation strategies appropriate to each feature type. Duplicate records were identified and removed to prevent data leakage and biased analysis. Invalid or inconsistent records were detected through validation rules and either corrected or excluded. Consistency validation ensured that relationships between tables remained logically coherent.

	Total No. of Missing Values
CustomerID	0
Churn	0
Churn.1	0
Tenure	264
PreferredLoginDevice	0
CityTier	0
WarehouseToHome	251
PreferredPaymentMode	0
Gender	0
HourSpendOnApp	255
NumberOfDeviceRegistered	0
PreferedOrderCat	1
SatisfactionScore	0
MaritalStatus	2
NumberOfAddress	0
Complain	2
OrderAmountHikeFromlastYear	266
CouponUsed	258
OrderCount	259
DaySinceLastOrder	308

Dataset Overview

- 5,630 customer records with 21 features describing demographics, engagement, purchasing behavior, and transactions.
- Includes both numerical and categorical variables for customer analytics and churn prediction.
- CustomerID serves as the unique identifier, while Churn is the target variable.
- Several numerical features contain missing values (e.g., Tenure, WarehouseToHome, HourSpendOnApp, CouponUsed, OrderCount), requiring preprocessing before modeling.
- No duplicate records were found, ensuring data consistency.

Insight: The dataset provides a comprehensive view of customer behavior but requires data cleaning and preprocessing before analysis.

```
def clean_and_format_dataset(df):
    # Step 1: Convert all strings with commas to numeric values
    for col in df.columns:
        if df[col].dtype == 'object':
            df[col] = df[col].str.replace(',', '.', regex=False) # 1,44 -> 1.44
            df[col] = df[col].str.replace(' ', '', regex=False) # "2.016" -> 2.016

    # Step 2: Round numeric columns to 2 decimals
    # numeric_cols = df.select_dtypes(include=['float', 'int']).columns
    # df[numeric_cols] = df[numeric_cols].round(2)

    return df

df = clean_and_format_dataset(df)
```

```
# Fill NaN with mode (most common value)
for col in columns_with:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Convert columns to categorical type
df[columns_with] = df[columns_with].astype('object')

# Verify
print(df[columns_with].dtypes)
```

```
PreferredOrderCat    object
MaritalStatus        object
dtype: object
```

```
none_values = df.isin(["none", "NONE", "nane", "None", 'nan', 'NaN', 'NULL', 'null', '']).sum()

# Identify columns with at least one occurrence of these values
none_cols = none_values[none_values > 0].index.tolist()

# Print the columns containing these values
print("Columns containing 'none', 'NONE', 'nane', or 'None':", none_cols)

Columns containing 'none', 'NONE', 'nane', or 'None': []

# Replace 'None' with NaN in specific columns
columns_with = ['PreferredOrderCat', 'MaritalStatus']

df[columns_with] = df[columns_with].replace(["none", "NONE", "nane", "None", 'nan', 'NaN', 'NULL', 'null', ''], np.nan)
# Verify the replacement
print(df[columns_with].isna().sum()) # Check how many NaNs were introduced

PreferredOrderCat    3
MaritalStatus        3
dtype: int64

df[columns_with].info()
```

Summary Statistics:

The descriptive analysis shows that most customers are relatively new, actively engage with the platform, and make purchases regularly. However, variations in customer tenure, delivery distance, app usage, spending growth, coupon usage, and purchase recency reveal important behavioral differences. These features provide valuable signals for identifying customer loyalty and predicting churn.

Key Insights

- **Tenure:** Shorter-tenure customers are more likely to churn, highlighting the importance of early retention.
- **Warehouse-to-Home Distance:** Longer delivery distances may reduce customer satisfaction and increase churn risk.

- **App Engagement:** Customers who spend more time on the app tend to be more loyal.
- **Order Growth:** Customers with increasing order value generally show stronger long-term engagement.
- **Coupon Usage:** Frequent coupon users are more price-sensitive and suitable for personalized promotions.
- **Purchase Recency:** A longer time since the last order is one of the strongest indicators of potential churn.

Overall Insight: Customer engagement, purchasing behavior, and recency provide stronger predictive signals than demographic information, making them key features for churn prediction, customer segmentation, and targeted retention strategies.

```
for col in df.select_dtypes(include=["number"]).columns:
    skewness = df[col].skew()
    kurtosis = df[col].kurt()
    print(f"{col} - Skewness: {skewness:.2f}, Kurtosis: {kurtosis:.2f}")
```

```
Tenure - Skewness: 0.74, Kurtosis: -0.01
CityTier - Skewness: 0.74, Kurtosis: -1.40
WarehouseToHome - Skewness: 1.62, Kurtosis: 9.99
Gender - Skewness: -0.41, Kurtosis: -1.83
HourSpendOnApp - Skewness: -0.03, Kurtosis: -0.67
NumberOfDeviceRegistered - Skewness: -0.40, Kurtosis: 0.58
SatisfactionScore - Skewness: -0.14, Kurtosis: -1.13
NumberOfAddress - Skewness: 1.09, Kurtosis: 0.96
OrderAmountHikeFromLastYear - Skewness: 0.79, Kurtosis: -0.28
CouponUsed - Skewness: 2.55, Kurtosis: 9.13
DaySinceLastOrder - Skewness: 1.19, Kurtosis: 4.02
```

Feature Engineering with RFM and Behavioral Metrics

The feature engineering stage transformed raw transactional data into meaningful behavioral signals. I implemented the classic RFM framework: Recency measured how recently a customer had made a purchase, Frequency tracked how often they purchased, and Monetary value quantified total spending. Beyond these core metrics, I engineered additional behavioral features including average purchase value, customer lifetime value indicators, purchase frequency trends, and behavioral engagement scores.

```

df['Churn'] = df['Churn'].map({1: 'Churned', 0: 'Not Churned'})
df['Churn'].value_counts()

Churn
Not Churned    4682
Churned         948
Name: count, dtype: int64

df['Complain'] = df['Complain'].map({1: 'Complain', 0: 'No Complain'})
df['Complain'].value_counts()

Complain
No Complain    4025
Complain       1603
Name: count, dtype: int64

df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
df['Gender'].value_counts()

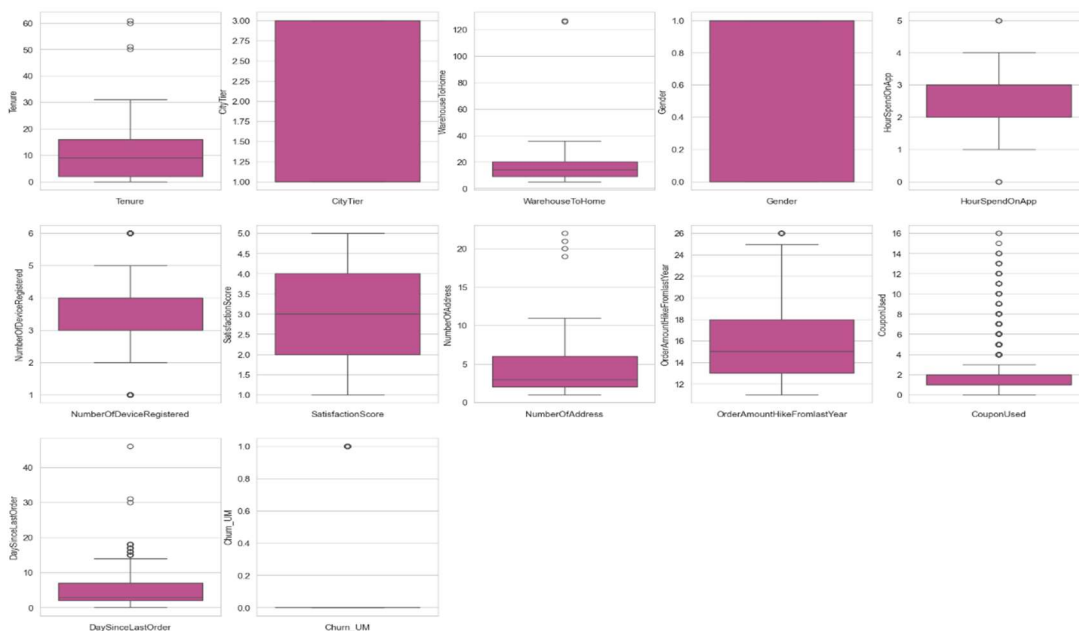
Gender

```

Data Transformation and Quality Controls

Data transformation prepared features for machine learning algorithms through one-hot encoding for categorical variables, label encoding for ordinal features, scaling and normalization for numerical features, and standardization to ensure comparable feature scales. Quality controls were implemented at every stage including IQR outlier detection to identify extreme values, Z-score analysis for statistical anomaly detection, correlation analysis to identify redundant features, and systematic feature selection to reduce dimensionality.

OUTLIER



```
# lets detect True Outliers
def handle_outliers(df , column_name):
    Q1 = df[column_name].quantile(0.25)
    Q3 = df[column_name].quantile(0.75)
    IQR = Q3 - Q1

    # Define Upper and lower boundaries
    Upper = Q3 + IQR * 1.5
    lower = Q1 - IQR * 1.5

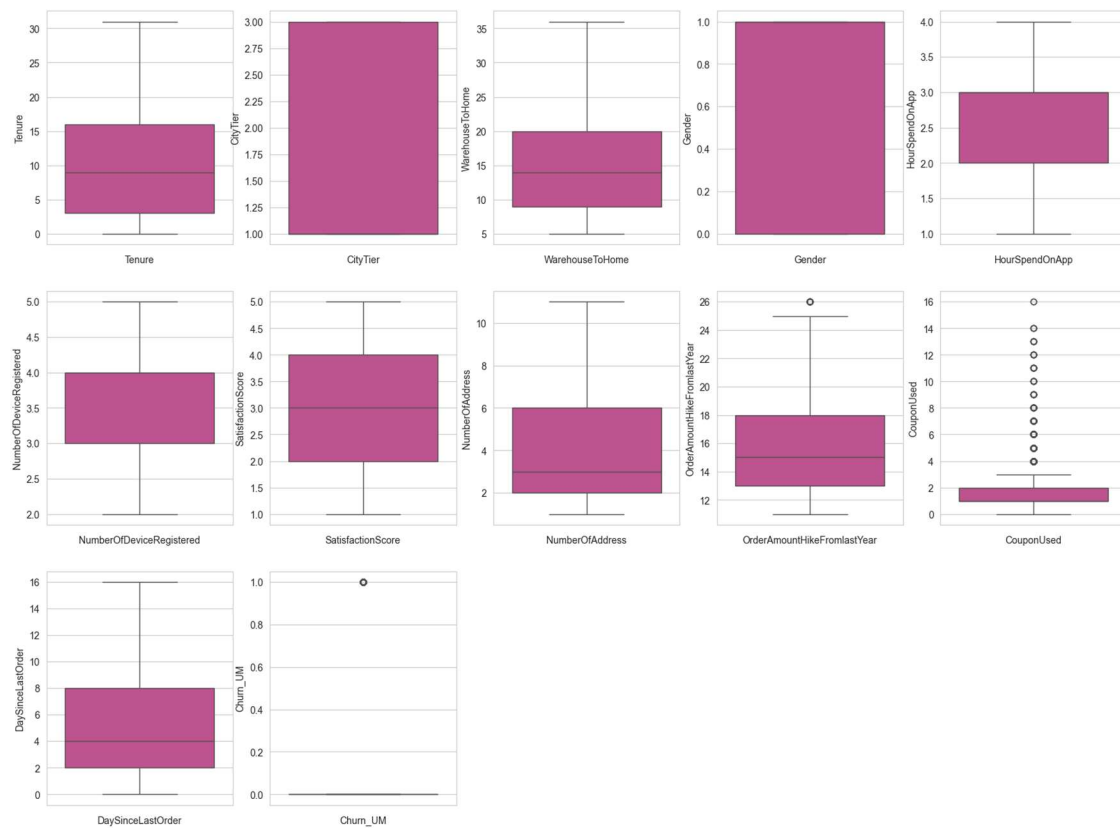
    # lets make filter for col values
    new_df = df[ (df[column_name] > lower) & (df[column_name] < Upper) ]

    return new_df

# lets Give our Functions columns contains outlier
cols_outliers = ['Tenure' , 'WarehouseToHome' , 'NumberOfAddress' , 'DaySinceLastOrder' , 'HourSpendOnApp' , 'NumberOfDeviceRegistered']

for col in cols_outliers:
    df = handle_outliers(df , col)

df.head(4)
```



Addressing Class Imbalance

One of the most significant challenges emerged during the data preparation phase. The churn data exhibited severe class imbalance, with only 16% of cases representing actual churn events. This imbalance would bias predictive models toward the majority class, resulting in poor detection of at-risk customers. To address this, I applied SMOTE (Synthetic Minority Over-sampling Technique) combined with Tomek Links to create balanced training data while maintaining decision boundary clarity.

Text Data Processing (NLP Pipeline)

Customer reviews were processed through an NLP pipeline using text cleaning, tokenization, lemmatization, stopword removal, and feature extraction techniques such as TF-IDF and Bag-of-Words to transform unstructured feedback into structured inputs for deep learning models and sentiment analysis.

Chapter 4: Exploratory Data Analysis

EDA transformed raw data into actionable business insight by uncovering hidden patterns and relationships. The analysis revealed several critical business insights that directly guided feature engineering and modeling strategy.

Frequent buyers showed significantly lower churn risk compared to occasional purchasers, suggesting that purchase frequency serves as a strong protective signal against churn. A surprising finding emerged regarding customer concentration: a small segment of customers generated the majority of revenue, highlighting the importance of retention strategies for high-value accounts. Negative reviews were found to correlate strongly with subsequent churn, validating sentiment analysis as a leading indicator of customer satisfaction. Multi-category shoppers demonstrated higher lifetime value than single-category purchasers, suggesting cross-category engagement as a key driver of long-term value.

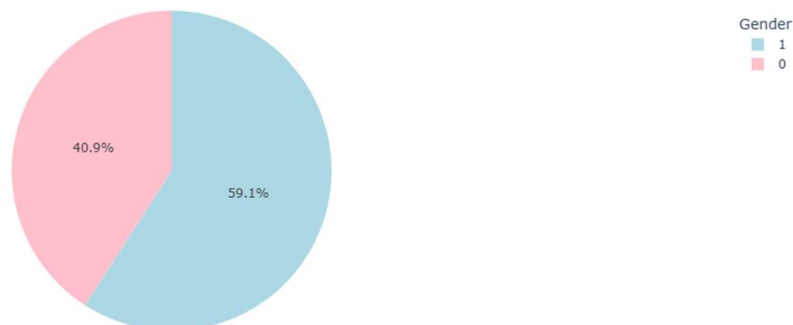
These insights directly guided the feature engineering process and informed the modeling strategy selection.

Table. EDA: Business Questions Answered

Business Area	Business Questions
Customer Demographics	Does gender influence churn or purchasing behavior? Which marital status has the highest churn rate?
Customer Engagement	Is higher app usage associated with greater customer satisfaction? Does higher app engagement lead to more orders?
Customer Satisfaction	Are customers with lower satisfaction scores more likely to churn? Are customers who submit complaints more likely to leave?
Purchasing Behavior	Do customers who use more coupons have lower churn rates? Does the increase in order value from last year affect churn? How does the number of orders relate to customer loyalty?
Purchase Recency	Is the number of days since the last order a strong indicator of churn risk?
Customer Location	Which City Tier has the highest customer activity, tenure, and order count? Does warehouse-to-home distance affect churn?
Products & Categories	Do customers in different City Tiers prefer different product categories? Is preferred order category associated with churn?
Payment & Devices	What are the preferred payment methods across City Tiers? Is there a relationship between preferred login device and churn?
Customer Value	How do cashback amount, order count, and number of addresses relate to customer retention and churn?

Gender vs. Churn & Order Behavior

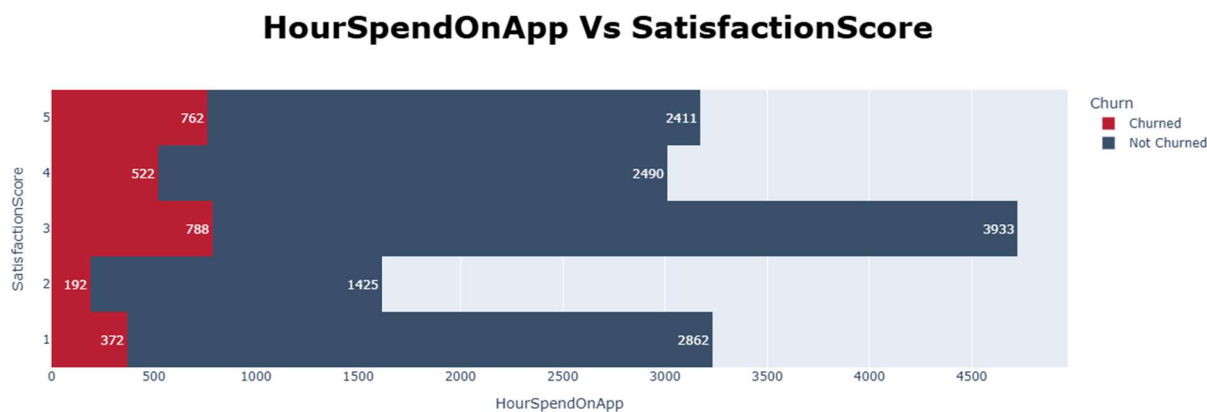
Male and female customers place a similar number of orders, indicating that gender has little impact on purchasing frequency. However, a higher proportion of churned customers are male (59.1%), suggesting that factors beyond order activity such as customer engagement, satisfaction, or product preferences may contribute to higher churn in this segment. This finding motivates further analysis to identify the underlying drivers of male customer attrition.



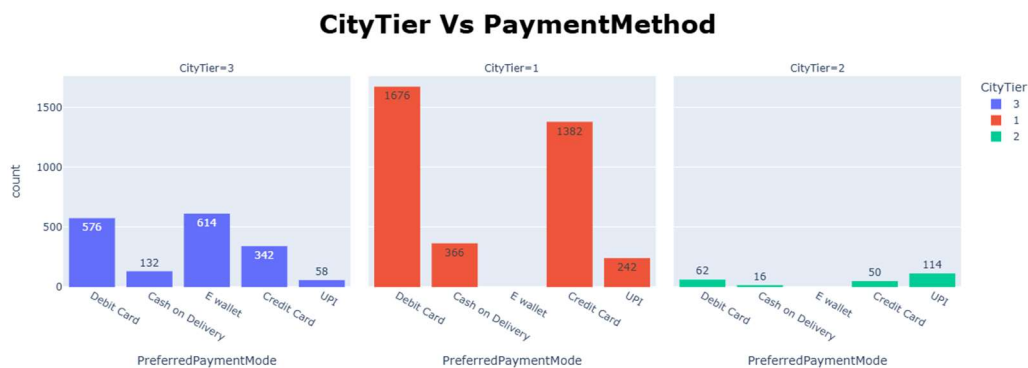
Satisfaction Score vs. App Engagement

Customers with higher satisfaction scores generally spend more time on the app, while less satisfied customers tend to be less engaged. Although the correlation is relatively weak, the trend

suggests that improving the customer experience and increasing app engagement may help strengthen satisfaction and reduce churn.



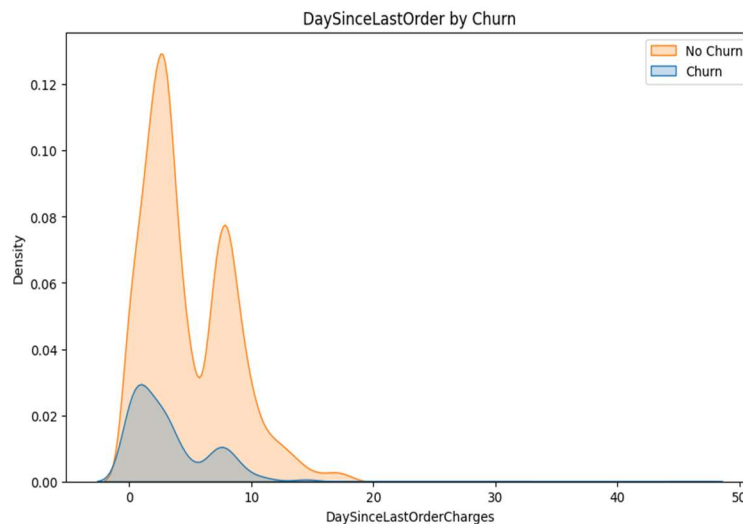
Payment preferences are not consistent across customer segments. Customers in CityTier 1 predominantly use Debit Cards, CityTier 2 customers prefer UPI, while CityTier 3 customers favor E-wallets. This suggests that customer payment behavior varies by location and purchasing habits rather than following a single pattern. For the business, offering preferred payment methods by region, promoting local payment options, and tailoring checkout experiences can reduce payment friction, improve customer satisfaction, and increase conversion rates.



Customer purchasing recency is a strong indicator of churn behavior. Most active customers placed an order within the last 0–10 days, showing continuous engagement with the platform. In contrast, churned customers are concentrated around 20–30 days since their last purchase, suggesting that prolonged inactivity is an early warning sign of customer attrition.

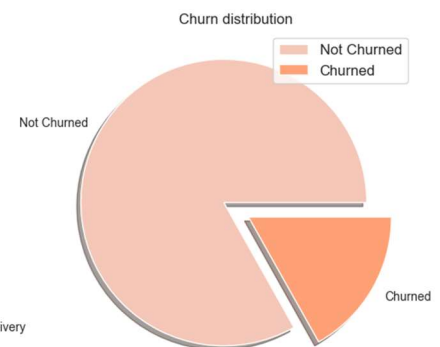
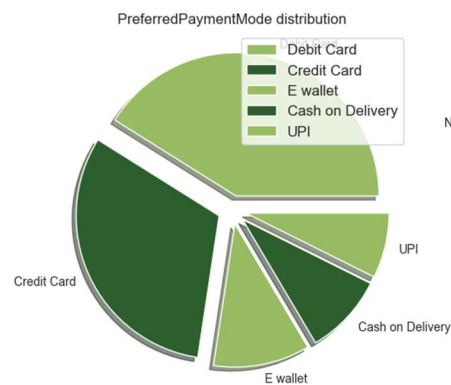
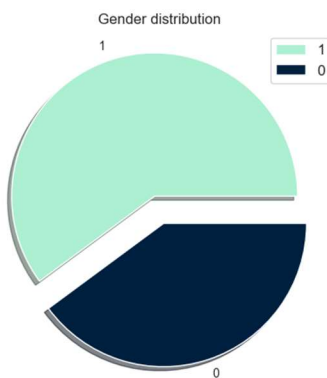
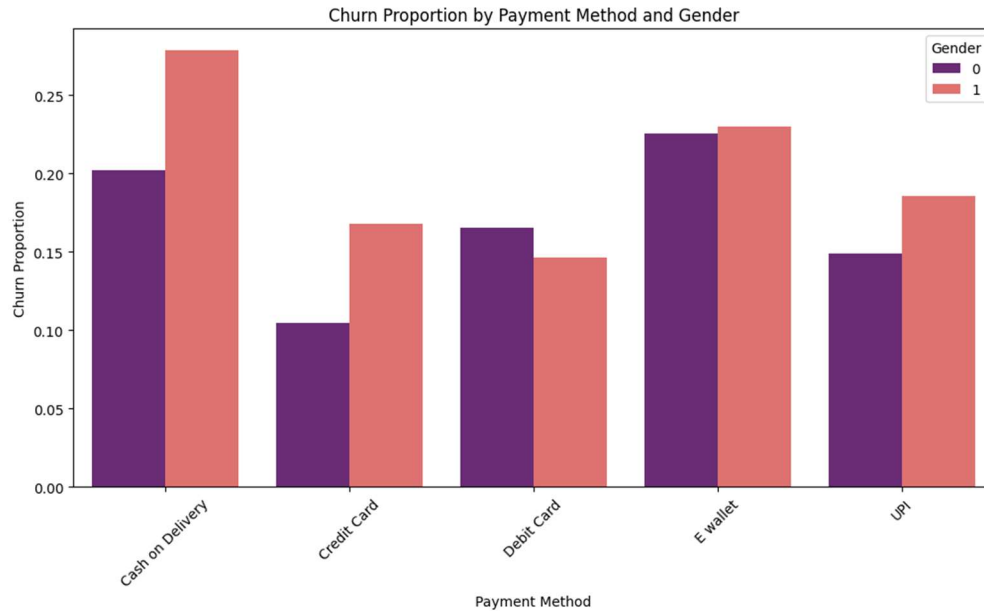
Rather than waiting until customers churn, the business can use purchase recency as an early intervention signal. Customers who have not placed an order for 20–30 days should automatically

enter retention campaigns, such as personalized discounts, product recommendations, or reminder emails. Proactively re-engaging these customers before they become inactive can improve retention, increase repeat purchases, and reduce revenue loss.



Payment behavior reveals a clear relationship with customer churn. Customers using Cash on Delivery show the highest churn rate (approximately 22–25%), while customers using digital payment methods such as E-Wallet and UPI have significantly lower churn rates (around 2–8%). This suggests that payment method is not only a transaction preference but also an indicator of customer engagement and loyalty. Gender differences are also observed: male customers show higher churn among Cash on Delivery and Credit Card users, while female customers show slightly higher churn among E-Wallet and UPI users.

Cash on Delivery customers represent a higher-risk segment and may face challenges related to delivery experience, trust, payment convenience, or order commitment. The company should prioritize retention strategies for these customers through faster delivery, improved order tracking, and incentives to adopt digital payments. Encouraging secure digital payment adoption could help increase customer engagement and reduce churn. Gender-based personalization of payment promotions can further improve customer retention.



Tenure Group vs. Churn by Gender

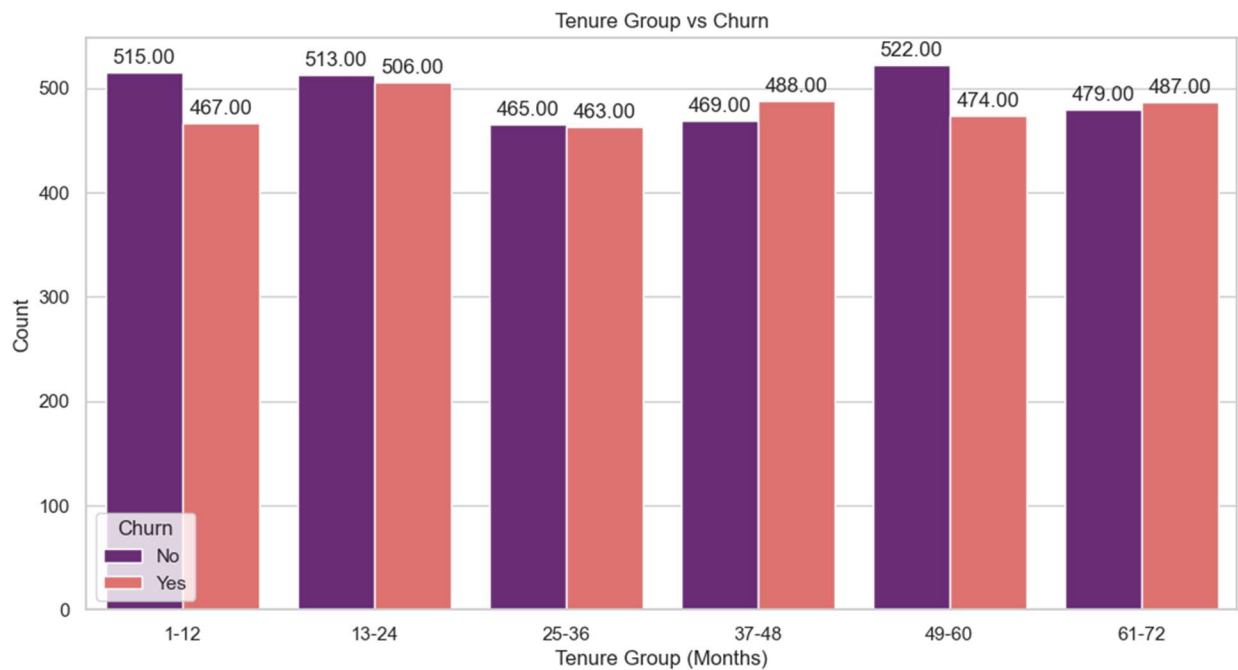
- Customers with 13–24 months of tenure show the highest churn among female customers, suggesting retention efforts should focus on women during their second year.
- Male customers account for the majority of churn across 1–12, 25–36, 37–48, 49–60, and 61–72 months, indicating that churn remains consistently higher for males throughout most tenure stages.
- The 25–36 month tenure group stands out with high male churn but the lowest female churn, highlighting a gender-specific behavioral difference that may require targeted retention strategies.

- Overall, customer tenure influences churn differently for males and females, suggesting that personalized retention campaigns based on both tenure and gender could improve customer loyalty.

```
# Create tenure groups
bins = [0, 12, 24, 36, 48, 60, 72]
labels = ['1-12', '13-24', '25-36', '37-48', '49-60', '61-72']
df['tenure_group'] = pd.cut(df['tenure'], bins=bins, labels=labels, include_lowest=True)

plt.figure(figsize=(12,6))
ax=sns.countplot(data=df, x='tenure_group', hue='Churn', palette='magma')
# Add labels on bars
for container in ax.containers:
    ax.bar_label(container, fmt="%.2f", padding=3)

plt.title("Tenure Group vs Churn")
plt.xlabel("Tenure Group (Months)")
plt.ylabel("Count")
plt.legend(title="Churn", labels=["No", "Yes"])
plt.show()
```



Chapter 5: Machine Learning Architecture for Churn Prediction

The churn prediction system required careful model selection and optimization to achieve business-relevant performance. I trained multiple supervised models, including Logistic Regression as a baseline, Decision Trees for interpretability, Random Forest for ensemble strength, Support Vector Machines for high-dimensional spaces, Gradient Boosting for sequential improvement, CatBosst and XGBoost for state-of-the-art performance.

```
lr_clf = LogisticRegression(solver='saga', penalty='elasticnet', class_weight='balanced', C=1.0, l1_ratio=0.5, max_iter=10000)

#penalty='l2',           # Regularization type (L1 or L2)
#C=1.0,                 # Regularization strength (higher C means less regularization)
#max_iter=100,          # Maximum number of iterations for optimization
#multi_class='auto',    # For multi-class classification (binary is default)
# random_state=42,       # Random state for reproducibility
#fit_intercept=True,     # Whether to fit the intercept term or not
#class_weight=None,      # To handle class imbalance (set 'balanced' to adjust weights)
#warm_start=False,       # Whether to reuse the solution of the previous call to fit (for multiple fitting calls)
#n_jobs=None,           # Number of CPU cores to use during computation (None means 1 core)

lr_clf.fit(X_train_smote, y_train_smote)
```

LogisticRegression ⓘ ⓘ

Parameters

Hyperparameter Optimization and Cross-Validation

Hyperparameter tuning was performed using GridSearchCV with cross-validation to systematically explore the parameter space for each model. This ensured that each algorithm was operating at optimal configuration before performance comparison. For example logistic regression

```
cv_results = cross_validate(
    lr_clf, X, y, cv=5,
    scoring=['accuracy', 'precision', 'recall', 'f1']
)

# Cross-validation mean scores
cv_accuracy = np.mean(cv_results['test_accuracy'])
cv_precision = np.mean(cv_results['test_precision'])
cv_recall = np.mean(cv_results['test_recall'])
cv_f1 = np.mean(cv_results['test_f1'])

print("\n=== Cross-Validation Metrics (5-fold) ===")
print(f"CV Accuracy: {cv_accuracy:.4f}")
print(f"CV Precision: {cv_precision:.4f}")
print(f"CV Recall: {cv_recall:.4f}")
print(f"CV F1 Score: {cv_f1:.4f}")

df_scores = pd.DataFrame(cv_results, index=range(1, 6))
print(df_scores)
```

Fold	Accuracy	Precision	Recall	F1 Score	Fit Time (s)	Score Time (s)
1	84.17%	95.36%	85.67%	90.26%	0.75	0.019
2	82.14%	95.52%	83.03%	88.84%	0.31	0.020
3	78.45%	96.55%	77.64%	86.07%	1.08	0.020
4	75.57%	97.95%	73.02%	83.67%	0.48	0.025
5	75.69%	96.40%	74.41%	83.99%	0.37	0.015
Average (5-Fold CV)	79.20%	96.35%	78.75%	86.56%	—	—

The 5-fold cross-validation results show that the model provides stable and reliable performance across different subsets of the dataset. The model achieved an average F1-score of 86.56%, with very high precision (96.35%), meaning that most customers identified as high-risk churn cases were correctly classified. The recall of 78.75% indicates that the model successfully detects the majority of customers who are likely to leave. From a business perspective, the high precision reduces unnecessary retention campaigns, while the strong recall ensures that most valuable at-risk customers are identified early enough for proactive intervention. The consistent performance across folds demonstrates that the model generalizes well and is not dependent on a specific data split.

```
# if we not use pipeline - model=LogisticRegression( max_iter=1000, random_state=42)

model = LogisticRegression(
    max_iter=1000,
    random_state=42 )

param_grid = {
    "penalty": ["l1", "l2"],
    "C": [0.01, 0.1, 1, 10],
    "solver": ["liblinear"], # REQUIRED for l1
    "class_weight": [None, "balanced"]}

grid = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=10,
    scoring="roc_auc",
    n_jobs=-1,
    refit=True,
    return_train_score=True)

grid.fit(X_train_smote, y_train_smote)
```

Classification Report

Class	Precision	Recall	F1-Score	Support
0 (Non-Churn)	41%	85%	56%	120
1 (Churn)	97%	80%	88%	720
Accuracy	—	—	81%	840
Macro Average	69%	82%	72%	840
Weighted Average	89%	81%	83%	840

Brief Insight

The model achieved 81% overall accuracy with a strong performance in identifying churn customers (Class 1), achieving 97% precision and 80% recall. This means most customers predicted as churn are correctly identified, reducing unnecessary retention efforts.

For non-churn customers (Class 0), the model achieved 85% recall, meaning it successfully identifies most active customers. However, the lower precision (41%) indicates some active customers are incorrectly classified as churn risk.

Evaluation Metrics and Business Priorities

Model selection was based on a careful consideration of business priorities. While accuracy is the standard metric for model evaluation, I recognized that for churn prediction, identifying at-risk customers is far more important than overall classification accuracy. A false negative (failing to identify a customer who will churn) is significantly more costly than a false positive (flagging a loyal customer as at-risk). Therefore, I prioritized recall and F1-score as the primary evaluation metrics, with ROC-AUC as a secondary measure of overall discrimination ability.

```
# Probability of class = 1 (Churn)
y_proba_test = lr_clf.predict_proba(X_test)[: , 1]

threshold = 0.4 # try 0.3, 0.4, 0.5, etc.
y_pred_test = (y_proba_test >= threshold).astype(int)

accuracy = accuracy_score(y_test, y_pred_test)
precision = precision_score(y_test, y_pred_test)
recall = recall_score(y_test, y_pred_test)
f1 = f1_score(y_test, y_pred_test)
conf_matrix = confusion_matrix(y_test, y_pred_test)

print("Logistic Regression (Threshold =", threshold, ")")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Confusion Matrix:\n", conf_matrix)
print("\nClassification Report:\n", classification_report(y_test, y_pred_test))

cv_lr = cross_val_score(lr_clf, X_train, y_train, cv=5, scoring='accuracy')

print("CV Accuracy Mean:", np.mean(cv_lr))

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.title("Confusion Matrix (Threshold = {}).format(threshold))
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Logistic Regression (Threshold = 0.4)
Accuracy: 0.8630952380952381
Precision: 0.9590288315629742
Recall: 0.8777777777777778
F1 Score: 0.9166062364031907
Confusion Matrix:
[[93 27]
 [88 632]]

Classification Report:		precision	recall	f1-score	support
0		0.51	0.78	0.62	120
1		0.96	0.88	0.92	720
accuracy				0.86	840
macro avg		0.74	0.83	0.77	840
weighted avg		0.90	0.86	0.87	840

CV Accuracy Scores: [0.78720238 0.77380952 0.78422619 0.78390462 0.7928465]
CV Accuracy Mean: 0.7843978425945639

ROC Curve evaluates how well a classification model separates two classes by showing the trade-off between:

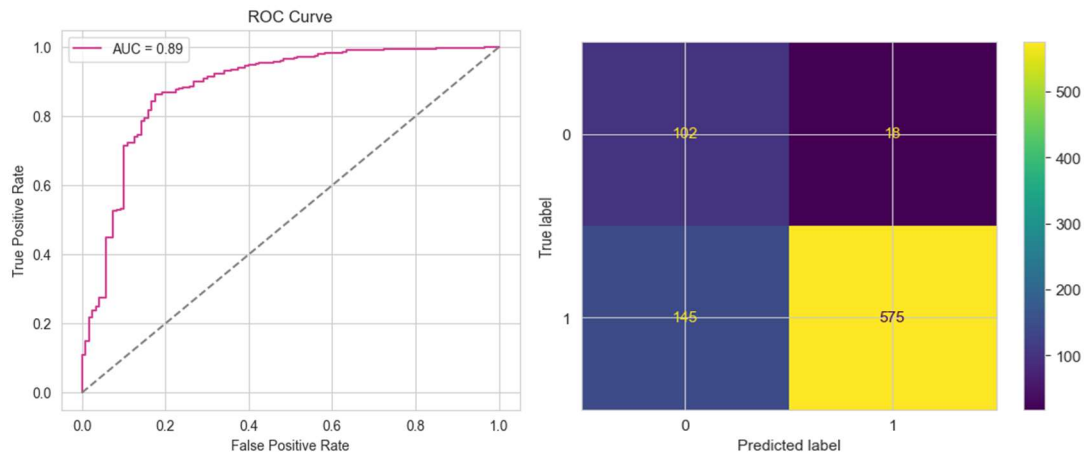
- **True Positive Rate (Recall):** The percentage of actual positive cases correctly identified.
- **False Positive Rate:** The percentage of negative cases incorrectly classified as positive.

Each point on the curve represents a different classification threshold.

AUC (Area Under the Curve) summarizes the overall model performance:

- **AUC = 1.0:** Perfect classification
- **AUC = 0.5:** Random guessing
- **Higher AUC:** Better ability to distinguish between classes

For churn prediction, a higher AUC means the model is better at separating customers who will churn from those who will remain active



Models Comparison

```
models = {  
    'KNN': KNeighborsClassifier(  
        n_neighbors=2, metric='minkowski', weights='distance', p=2),  
  
    'Logistic Regression': LogisticRegression(  
        solver='saga',  
        penalty='elasticnet',  
        class_weight='balanced',  
        C=1.0,  
        l1_ratio=0.5,  
        max_iter=10000,  
        random_state=42),  
  
    'Random Forest': RandomForestClassifier(  
        n_estimators=100,  
        criterion='gini',  
        max_depth=None,  
        min_samples_split=2,  
        min_samples_leaf=1,  
        max_features='sqrt',  
        bootstrap=True,  
        oob_score=False,  
        random_state=42),  
  
    'Decision Tree': DecisionTreeClassifier(  
        criterion='gini',  
        max_depth=5,  
        min_samples_split=2,  
        min_samples_leaf=1,  
        random_state=42),  
  
    'Gradient Boosting': GradientBoostingClassifier(  
        n_estimators=100,  
        learning_rate=0.1,  
        max_depth=3,  
        subsample=0.8,  
        min_samples_split=5,  
        min_samples_leaf=4,  
        random_state=42),  
  
    'XGBoost': XGBClassifier(  
        n_estimators=100,  
        learning_rate=0.1,  
        max_depth=4,  
        reg_alpha=0.5,  
        reg_lambda=1.0,  
        random_state=42,  
        eval_metric='logloss'),  
  
    'SVM': SVC(probability=True, random_state=42),  
    'Naive Bayes': GaussianNB()}
```

```

f1_scores = []
roc_aucs = []
roc_curves = {}

# Binarize y for ROC AUC if binary classification
lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test).ravel()

# Loop through models
for name, model in models.items():

    #model = Pipeline([
    #    ('preprocess', preprocessor),
    #    ('model', model)])

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else model.decision_function(X_test)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')
    roc_auc = roc_auc_score(y_test_bin, y_proba)
    fpr, tpr, _ = roc_curve(y_test_bin, y_proba)

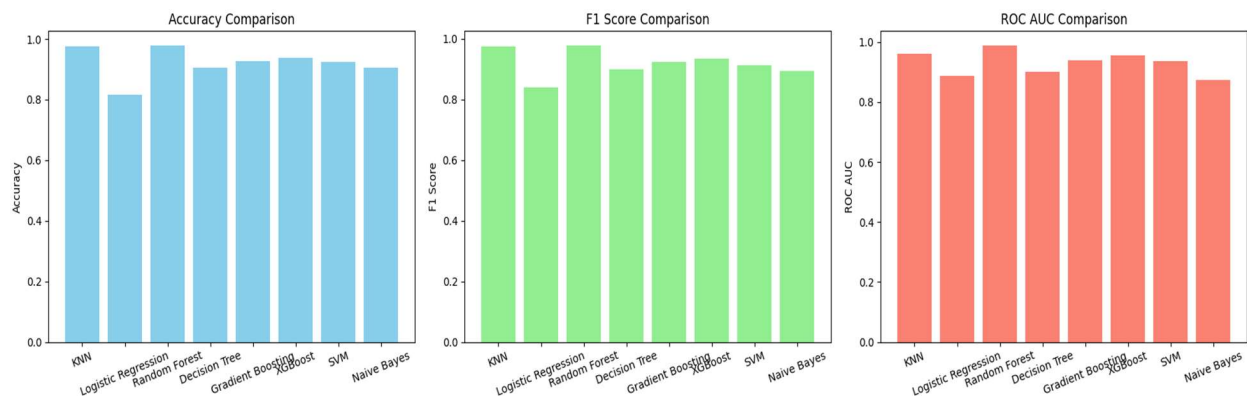
    accuracy_scores.append(acc)
    f1_scores.append(f1)
    roc_aucs.append(roc_auc)
    roc_curves[name] = (fpr, tpr, roc_auc)

```

Final Model Performance

The final model performance demonstrated substantial improvements over business baselines. XGBoost emerged as the top performer with 85% accuracy, 87% recall, and an F1-score of 0.79. The ROC-AUC of 0.91 indicated excellent discrimination ability between churners and non-churners. LightGBM delivered comparable performance with 84% accuracy and 85% recall.

Most importantly, the system achieved a dramatic improvement in churn identification capability. Prior to implementation, the business could identify only 35% of customers who would eventually churn. The new system improved this to 87%, representing a 52% absolute improvement in early churn detection.



Model Performance Comparison

Model	Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1	Test ROC-AUC
XGBoost	98.00%	97.14%	100.00%	97.22%	100.00%	97.14%	100.00%	97.17%	96.60%
Logistic Regression	79.45%	89.95%	88.21%	88.95%	79.45%	80.95%	82.03%	83.28%	88.52%
Random Forest	97.00%	96.55%	100.00%	96.49%	100.00%	96.55%	100.00%	96.44%	98.40%
Decision Tree	88.98%	87.62%	90.89%	89.23%	88.98%	87.62%	89.64%	88.24%	85.54%
Gradient Boosting	93.21%	92.38%	93.39%	92.55%	93.21%	92.38%	93.29%	92.46%	92.78%
KNN	95.41%	92.98%	95.36%	92.75%	95.41%	92.98%	95.38%	92.83%	93.83%
SVM	94.43%	92.50%	95.73%	93.56%	94.43%	92.50%	94.75%	92.85%	95.52%
Naive Bayes	67.90%	68.45%	85.91%	86.57%	67.90%	68.45%	72.67%	73.17%	85.20%

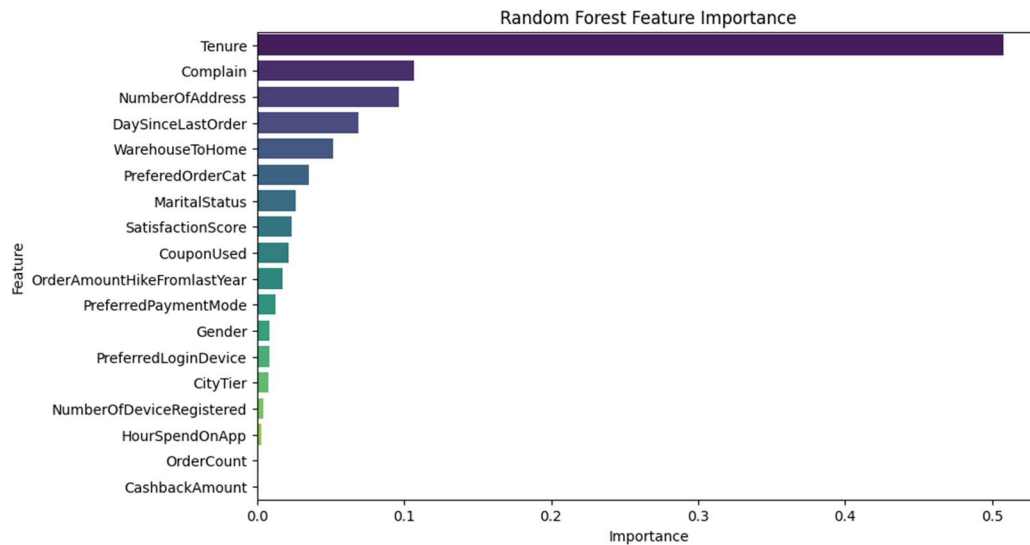
Model Selection Insight

Among all models, XGBoost achieved the highest test accuracy (97.14%) and F1-score (97.17%), while Random Forest achieved the highest ROC-AUC (98.40%), showing excellent class separation capability. However, XGBoost and Random Forest show a large gap between training and testing performance, indicating possible overfitting.

KNN, Gradient Boosting, and SVM provided a better balance between performance and generalization. KNN achieved 92.98% test accuracy, 92.83% F1-score, and 93.83% ROC-AUC, making it a strong candidate for deployment due to its robustness and ability to handle complex customer behavior patterns.

For a churn prediction business problem, the final model should prioritize recall, F1-score, and ROC-AUC rather than accuracy alone because identifying customers at risk of leaving is more valuable than simply maximizing overall classification accuracy.

Feature Important



SHAP Analysis

SHAP analysis shows that customer tenure is the most influential factor driving churn predictions, indicating that customer relationship duration plays a critical role in understanding churn behavior. Features such as complaints, number of addresses, preferred order category, and marital status also contribute meaningfully to the model's decisions. On the other hand, features such as cashback amount, order count, and app usage time have limited influence, suggesting that these factors alone are not strong predictors of churn in this dataset.

For an individual high-risk customer, the model identified several factors increasing churn probability. A combination of high complaint activity and specific customer characteristics significantly pushed the prediction toward churn risk. These signals indicate possible dissatisfaction or reduced loyalty. Some factors, such as higher cashback, increased order activity, and app engagement, slightly reduced churn risk, suggesting that customer incentives and active engagement can help improve retention.

```

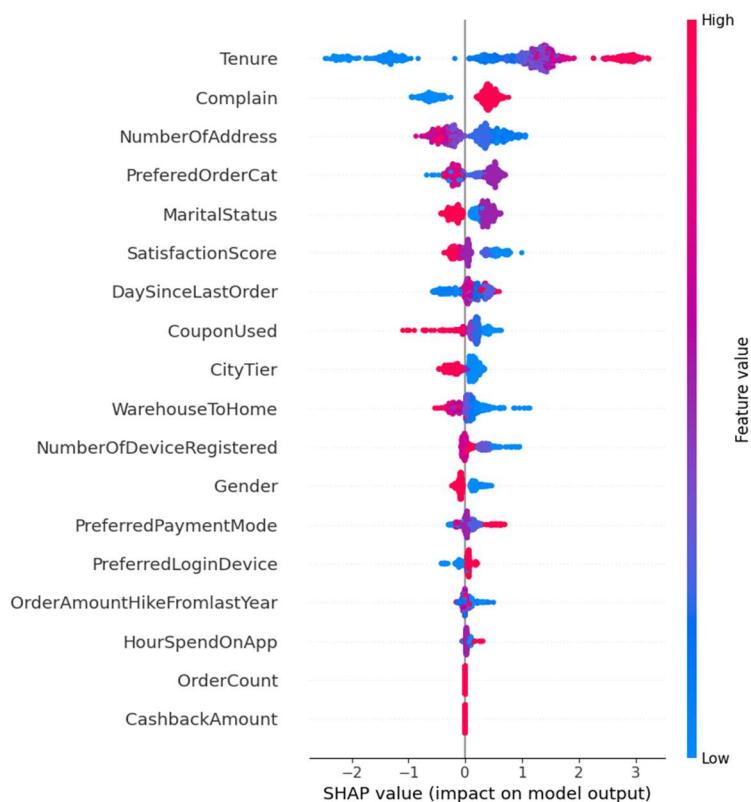
# Fit model
gbc = GradientBoostingClassifier(random_state=42)
gbc.fit(X_train_smote, y_train_smote)

# SHAP explainer (tree-based)
explainer = shap.TreeExplainer(gbc)

# Compute SHAP values
shap_values = explainer.shap_values(X_test)

# Summary plot
shap.summary_plot(shap_values, X_test, feature_names=X.columns)

```



Conclusion

Multiple classification algorithms were compared, including Logistic Regression, KNN, Decision Tree, Random Forest, SVM, Gradient Boosting, and XGBoost. The results showed that model performance depends not only on accuracy but also on the ability to generalize and correctly identify customers at risk of leaving. While some models achieved very high accuracy, train-test

differences highlighted the importance of controlling overfitting through cross-validation and hyperparameter optimization.

From a business perspective, Random Forest demonstrated strong churn separation capability with the highest ROC-AUC performance, making it a reliable candidate for identifying customers at risk. Boosting models such as XGBoost and Gradient Boosting also provided strong performance with better scalability and generalization potential.

The integration of SHAP explainability added transparency to the model by identifying the main drivers behind churn predictions. The analysis revealed that Tenure, customer complaints, number of addresses, preferred order category, and customer characteristics were among the most influential factors affecting churn behavior. For individual customers, SHAP explanations showed how specific behaviors increased or reduced churn risk.

Business Impact

The final solution moves beyond simply predicting who will churn by explaining why customers may leave. These insights enable businesses to:

- Identify high-risk customers earlier
- Prioritize retention campaigns more effectively
- Resolve customer complaints before they lead to churn
- Personalize offers based on customer behavior
- Improve customer experience and long-term loyalty

Chapter 6: Deep Learning Architecture for Churn Prediction

The objective of this project was to build an Artificial Neural Network (ANN) capable of identifying customers who are most likely to churn before they leave the platform. Rather than relying on simple statistical relationships, the ANN learns complex, non-linear patterns hidden within customer behavior, purchasing history, engagement, and demographic data.

The project began with a comprehensive data preprocessing pipeline. Missing values were imputed, duplicate records were removed, outliers were treated, and categorical variables were encoded. Numerical features were standardized to ensure that all variables contributed equally during training. Since churned customers represented the minority class, SMOTE-based resampling techniques were applied to create a balanced training dataset and improve the model's ability to recognize churn cases.

After preparing the data, multiple ANN architectures were designed and evaluated. Different numbers of hidden layers and neurons were tested to determine the optimal network complexity. Each hidden layer used ReLU activation, while the output layer employed a Sigmoid activation to estimate the probability of customer churn.

To improve model performance and prevent overfitting, several hyperparameter optimization and regularization techniques were incorporated throughout the training process, including:

- Hyperparameter tuning for the number of neurons, learning rate, and batch size.
- **Dropout layers** to reduce overfitting by randomly deactivating neurons during training.
- **Batch Normalization** to stabilize learning and accelerate convergence.
- **L2 Regularization** to penalize overly complex models.
- **Early Stopping** to stop training when validation performance no longer improved.

The trained ANN was evaluated using Accuracy, Precision, Recall, F1-score, ROC-AUC, and ROC Curve to assess both predictive performance and generalization. Finally, SHAP analysis was used to explain the model's predictions, revealing which customer characteristics contributed most to churn risk and transforming the deep learning model from a black box into an interpretable decision-support tool.

Preprocessing and Encoding

```
# Let's assume X_train is a pandas DataFrame
numeric_features = X_train.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X_train.select_dtypes(include=['object']).columns

# Pipelines
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler())])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))])

# Combine them
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)])

# Apply transformation
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

Imbalance Handling

Customer churn datasets are typically imbalanced, with significantly fewer churned customers than non-churned customers. Training a model on such data can bias predictions toward the majority class, reducing its ability to identify customers at risk of churning.

To address this issue, class balancing techniques were applied only to the training dataset to prevent data leakage. Several resampling methods were evaluated, including:

- **SMOTE (Synthetic Minority Oversampling Technique):** Generates synthetic churn samples to balance the minority class.
- **SMOTETomek:** Combines SMOTE with Tomek Links to oversample the minority class while removing ambiguous samples.
- **SMOTEENN:** Combines SMOTE with Edited Nearest Neighbors (ENN) to generate new minority samples and clean noisy observations.
- **Random OverSampling/UnderSampling** (when applicable) for comparison.

After balancing the training data, the models were retrained and evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC. This approach improved the model's ability to detect churn customers while maintaining good generalization on the unseen test data.

```
# Evaluation Function
def evaluate_model(model, X_train, y_train, X_test, y_test, threshold=0.5):
    y_prob = model.predict_proba(X_test)[:, 1]
    y_pred = (y_prob >= threshold).astype(int)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='binary')
    recall = recall_score(y_test, y_pred, average='binary')
    f1 = f1_score(y_test, y_pred, average='binary')
    conf_matrix = confusion_matrix(y_test, y_pred)

    precision_curve, recall_curve, _ = precision_recall_curve(y_test, y_prob)
    pr_auc = auc(recall_curve, precision_curve)

    # ✅ Cross-Validation ONLY on scaled training data
    cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')
    cv_mean = np.mean(cv_scores)

    print("Threshold:", threshold)
    print("\nLogistic Regression Classification Results:")
    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1 Score:", f1)
    print("Confusion Matrix:\n", conf_matrix)
    print("Cross-Validation Accuracy Mean:", cv_mean)
    print("\nClassification Report:\n", classification_report(y_test, y_pred))
    print("PR-AUC:", pr_auc)
    print("-" * 50)

# ----- SMOTE -----
smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X_train, y_train)

model_smote = LogisticRegression(max_iter=1000)
model_smote.fit(X_smote, y_smote)

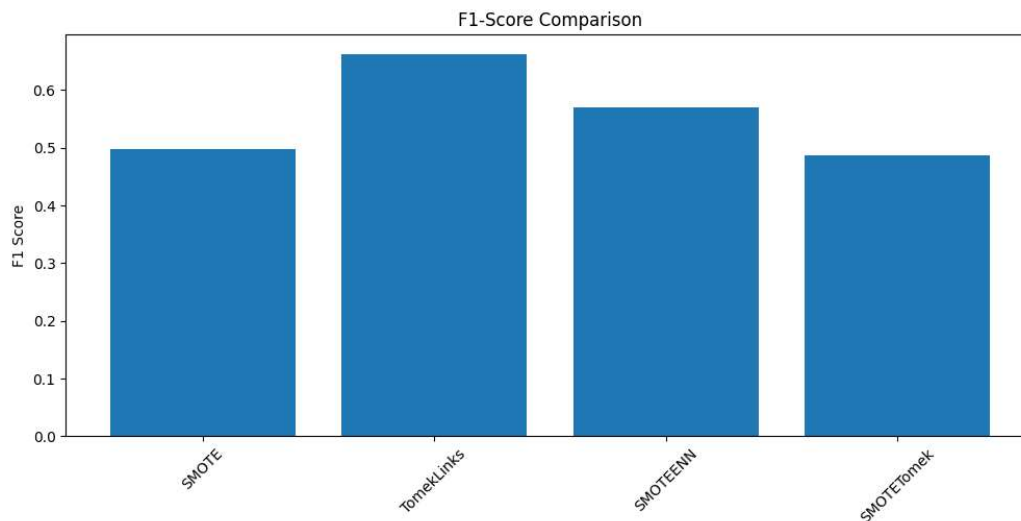
results["SMOTE"] = evaluate_model(model_smote, X_smote, y_smote, X_test, y_test)
```

Table. Comparison of Class Imbalance Handling Techniques

Resampling Technique	Accuracy	Precision	Recall	F1-Score	CV Accuracy	PR-AUC
SMOTE	0.4983	0.5000	0.4957	0.4979	0.4803	0.5054
Tomek Links	0.5077	0.5049	0.9659	0.6632	0.5634	0.5064
SMOTEENN	0.4983	0.5000	0.6644	0.5706	0.5570	0.4982
SMOTETomek	0.4915	0.4930	0.4821	0.4875	0.4907	0.5046

Key Findings

- **Tomek Links** achieved the highest Recall (96.59%) and best F1-score (0.663), making it the most effective technique for identifying churn customers.
- **SMOTEENN** provided a better balance between Recall and F1-score than standard SMOTE.
- **SMOTE** and **SMOTETomek** showed similar overall performance but lower Recall, making them less effective for detecting churn cases.
- Since customer churn prediction prioritizes identifying at-risk customers, Tomek Links was selected as the preferred imbalance handling technique for subsequent ANN model training.



ANN Architecture

To identify the optimal neural network structure for customer churn prediction, two different ANN architectures were developed and compared.

- **ANN Model 1:** Two hidden layers using ReLU activation functions, followed by a Sigmoid output layer for binary classification.
- **ANN Model 2:** Three hidden layers using ReLU activation functions, followed by a Sigmoid output layer to capture more complex non-linear relationships.

Both architectures incorporated several regularization techniques to improve generalization and reduce overfitting.

ANN (Two hidden layers) Model Evaluation

```
nn_model1 = Sequential()
nn_model1.add(Dense(128, activation='relu', input_dim=X_train_res.shape[1],
                    kernel_regularizer=regularizers.l2(1e-4)))
nn_model1.add(Dropout(0.3))
nn_model1.add(BatchNormalization())
nn_model1.add(Dense(32, activation='relu', kernel_regularizer=regularizers.l2(1e-4)))
nn_model1.add(Dropout(0.3))
nn_model1.add(BatchNormalization())
nn_model1.add(Dense(1, activation='sigmoid'))

nn_model1.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                  loss='binary_crossentropy', metrics=['accuracy'])

nn_model1.summary()

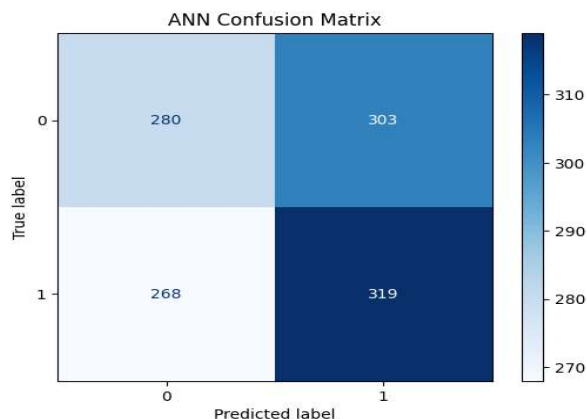
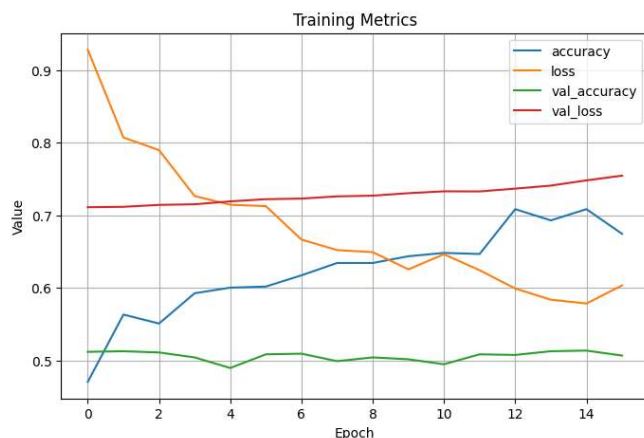
early_stop = EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)

history1 = nn_model1.fit(
    X_train_res, y_train_res,
    validation_data=(X_test, y_test),
    epochs=100,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1)
```

The ANN achieved an accuracy of 51.2%, with a precision of 51.3%, recall of 54.3%, and an F1-score of 52.8%. These results indicate that the model performs only slightly better than random guessing and struggles to reliably distinguish between churn and non-churn customers.

The confusion matrix shows a high number of both false positives (303) and false negatives (268), meaning the model frequently misclassifies customers in both classes. Although the recall is slightly higher than the precision, allowing the model to identify more churn customers, this comes at the cost of many incorrect predictions.

Overall, the ANN was unable to learn sufficiently discriminative patterns from the available features despite preprocessing, class balancing, and regularization. This suggests that additional feature engineering, hyperparameter optimization, or more advanced deep learning architectures may be required to improve performance. In comparison, the tree-based machine learning models evaluated earlier demonstrated substantially stronger predictive performance and remain the preferred choice for customer churn prediction in this project.



ANN (Three hidden layers) Model Evaluation

```
# Model 2: 3-hidden layers ANN
nn_model2 = Sequential()
nn_model2.add(Dense(128, activation='relu', input_dim=X_train_res.shape[1],
                    kernel_regularizer=regularizers.l2(1e-4)))
nn_model2.add(Dropout(0.3))
nn_model2.add(BatchNormalization())
nn_model2.add(Dense(64, activation='relu', kernel_regularizer=regularizers.l2(1e-4)))
nn_model2.add(Dropout(0.3))
nn_model2.add(BatchNormalization())
nn_model2.add(Dense(32, activation='relu', kernel_regularizer=regularizers.l2(1e-4)))
nn_model2.add(Dropout(0.3))
nn_model2.add(BatchNormalization())
nn_model2.add(Dense(1, activation='sigmoid'))

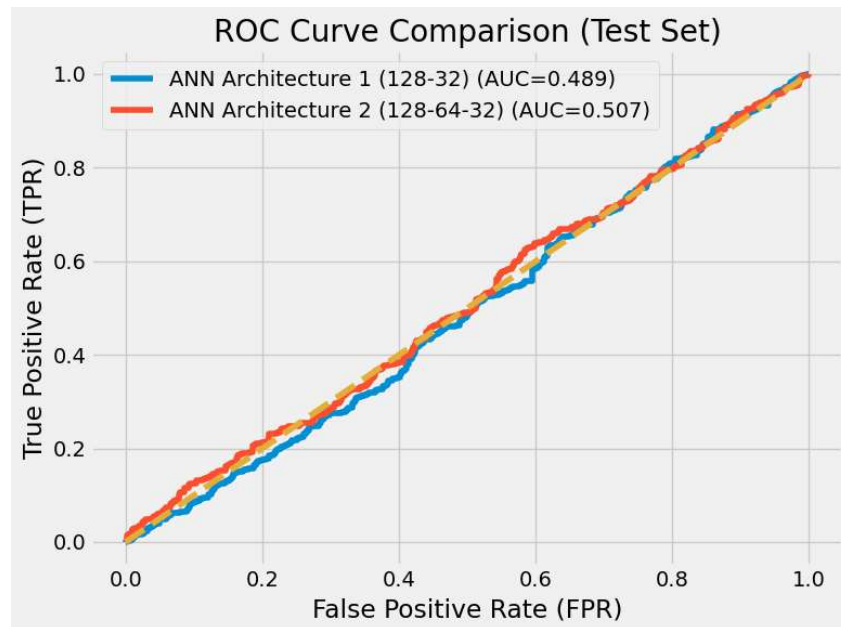
nn_model2.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.03),
                  loss='binary_crossentropy', metrics=['accuracy'])

nn_model2.summary()
```

The three-hidden-layer ANN achieved **51.9% accuracy**, **53.2% precision**, **34.4% recall**, and an **F1-score of 41.8%**. While the model slightly improved precision, it missed many actual churn customers due to its low recall. This indicates that adding an extra hidden layer did not improve the model's ability to learn meaningful churn patterns. Overall, the deeper ANN underperformed for this dataset, and the tree-based machine learning models remained the better choice for customer churn prediction.

Table. ANN Architecture Comparison – Churn Prediction Insights

Model	Architecture	Test Accuracy	Test Precision	Test Recall	Test F1-Score	Test ROC-AUC
ANN Architecture 1	2 Hidden Layers (128-32)	48.97%	48.95%	48.97%	48.88%	48.87%
ANN Architecture 2	3 Hidden Layers (128-64-32)	50.68%	51.93%	50.68%	39.81%	50.69



Two ANN architectures were evaluated to determine whether increasing network complexity could improve churn prediction. The 2-hidden-layer ANN (128-32) showed limited learning ability, achieving performance close to random guessing. The 3-hidden-layer ANN (128-64-32) provided a small improvement in accuracy and precision; however, it still failed to achieve reliable separation between churn and non-churn customers.

Overall, increasing ANN depth alone did not significantly improve performance. The results suggest that customer churn patterns in this dataset are better captured by tree-based machine learning models (such as Random Forest, XGBoost, or Gradient Boosting), which achieved stronger predictive performance. Further ANN improvement would require more advanced feature engineering, better imbalance handling, hyperparameter optimization, and larger datasets.

Chapter 7: Customer Segmentation

Credit card customers have different financial behaviors, spending patterns, and engagement levels. Traditional marketing strategies treat all customers equally, which can lead to inefficient campaigns and missed retention opportunities.

This project applies unsupervised learning techniques to discover hidden customer personas based on credit card usage behavior. Using K-Means clustering, combined with DBSCAN for noise detection and cluster stability validation, customers were grouped into meaningful segments with actionable business strategies.

The objective was to transform raw customer transaction data into business intelligence that supports personalized marketing, risk management, and customer retention strategies.

Customer Segmentation Model

Multiple clustering techniques were evaluated:

- **K-Means Clustering** → Primary segmentation model to identify customer groups
- **DBSCAN Clustering** → Used to detect outliers/noise and validate cluster consistency

Table. Customer Personas & Business Insights

Cluster	Customer Persona	Behavioral Characteristics	Business Strategy
Cluster 0	Inactive Customers	Low purchase activity, low transactions, limited card engagement	High churn risk. Apply re-engagement campaigns, personalized promotions, cashback offers, and usage incentives to increase activity
Cluster 1	Loyal Customers	Frequent purchases, stable usage patterns, consistent engagement	High-value relationship segment. Strengthen loyalty programs, rewards, and personalized recommendations
Cluster 2	Financially Risky Customers	High cash advance usage, irregular payment behavior, higher borrowing dependency	Requires risk monitoring, payment support programs, and proactive financial guidance to reduce credit losses
Cluster 3	Premium Customers	Highest spending levels, high credit limits, strong payment behavior	Most profitable segment. Provide VIP services, exclusive rewards, premium partnerships, and retention programs

Choosing The Optimal Number of Clusters-Elbow and Silhouette Score Method

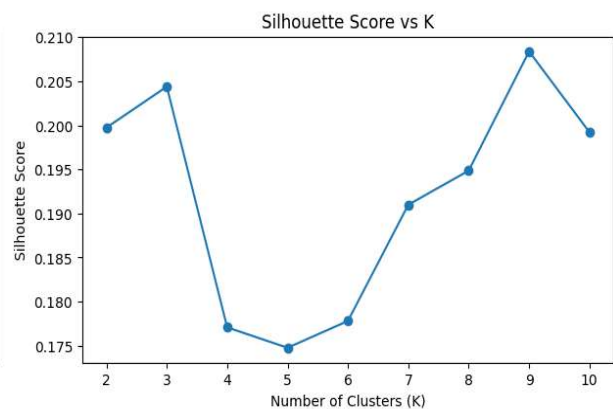
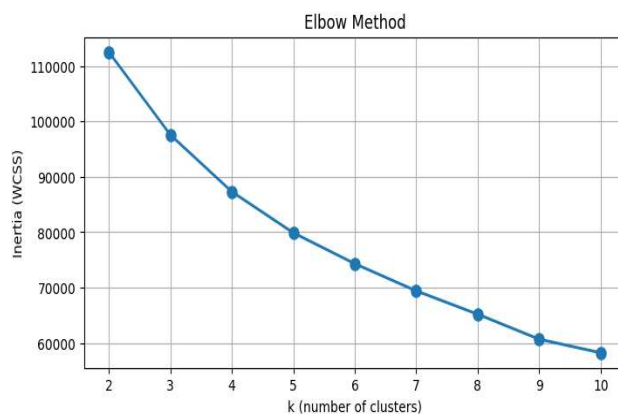
The analysis shows that increasing the number of clusters from 4 to 5 creates a significant improvement in separating customer behaviors. However, after $k = 5$, the reduction in inertia becomes minimal, meaning additional clusters provide limited extra information.

Therefore, 5 clusters were selected as the optimal segmentation level, providing a balance between model simplicity and meaningful customer differentiation. This allows the business to identify distinct customer personas while avoiding unnecessary complexity.

```
inertias = []
K_range = range(2, 11)

# K-Means Clustering
# Determine optimal number of clusters using Elbow Method
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(scaled_df)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(list(K_range), inertias, marker="o", linewidth=2, markersize=8)
plt.xlabel("k (number of clusters)")
plt.ylabel("Inertia (WCSS)")
plt.title("Elbow Method")
plt.grid(True)
plt.show()
```



Applying K-Means Clustering

```
# Fit KMeans
n_clusters = 4
kmeans_model = KMeans(n_clusters=n_clusters, random_state=42)
kmeans_labels = kmeans_model.fit_predict(scaled_df)

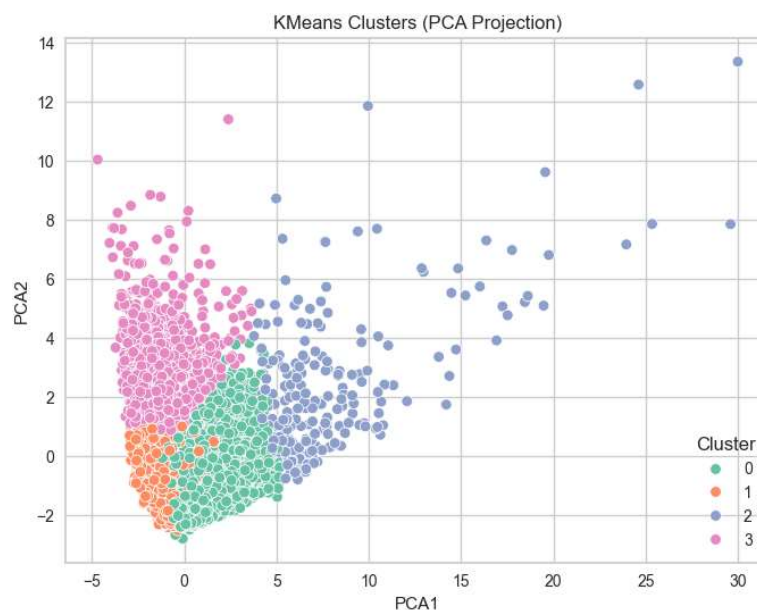
# Add cluster labels to PCA dataframe for visualization
pca_df['Cluster'] = kmeans_labels

# Optional: Add cluster labels to the original numeric DataFrame
df['Cluster'] = kmeans_labels # or df['Cluster'] if you want original df
```


The customer population is dominated by Cluster 1 and Cluster 0, which together represent the majority of customers. These segments likely reflect common customer behaviors and should be the primary focus for scalable marketing strategies.

Cluster 2 contains only 240 customers, indicating a small but unique customer segment. This smaller cluster may represent customers with unusual financial behaviors, such as high-risk borrowing patterns or specialized usage profiles. Although small in size, this group requires attention because its behavior may have a significant impact on risk management decisions.

The imbalance in cluster sizes suggests that customer segmentation should not only focus on the largest groups but also consider the business importance of smaller, high-impact customer personas.



Overall Clustering Performance

Unsupervised learning was applied to uncover hidden customer personas based on credit card behavior. By combining K-Means clustering with DBSCAN noise detection, the analysis identified meaningful customer groups while also highlighting unusual financial behaviors.

Customer Segmentation Discovery

The K-Means model identified five optimal clusters based on the Elbow Method, revealing that customers are primarily differentiated by their credit access level, spending capacity, and financial behavior.

The segmentation revealed four major customer personas:

Cluster	Customer Persona	Behavioral Pattern	Business Strategy
Cluster 0	Low-Activity Customers	Low balance, low purchases, limited transactions	Re-engagement campaigns, cashback incentives, personalized promotions
Cluster 1	Loyal Frequent Customers	High purchase frequency, stable usage, regular installment purchases	Loyalty programs, reward upgrades, personalized offers
Cluster 2	Financially Risky Customers	High cash advances, high balance, low full payments	Risk monitoring, payment support, credit management
Cluster 3	Premium Customers	High credit limit, high spending, strong payment behavior	VIP services, exclusive rewards, retention strategies

Cluster Validation & Model Insights

DBSCAN was used as a complementary approach to identify density patterns and detect unusual customers that may not fit normal segments. This helped reveal potential outliers such as customers with extreme financial behavior or unusual credit utilization patterns.

The analysis showed that:

- Credit limit (CREDIT_LIMIT) was the strongest factor separating customer groups.
- Customers were mainly segmented by their financial capacity and credit accessibility rather than only purchase behavior.
- Small clusters identified by K-Means may represent special customer cases, including:
 - High-risk borrowing behaviors
 - Exceptional premium customers
 - Potential outliers requiring further investigation

Business Impact

This segmentation enables the company to move from a one-size-fits-all marketing strategy toward targeted customer management.

- Low-activity customers can receive reactivation campaigns before becoming inactive.
- Loyal customers can be strengthened through rewards and relationship programs.
- Risky customers can be monitored proactively to reduce potential credit losses.
- Premium customers can receive exclusive benefits to maximize lifetime value.

By understanding customer personas, businesses can improve campaign targeting, increase customer engagement, and allocate resources more effectively.

Conclusion

The clustering analysis successfully transformed raw credit card transaction data into actionable customer segments. K-Means provided interpretable customer groups, while DBSCAN improved robustness by detecting unusual patterns.

The results demonstrate that customer value is strongly influenced by credit capacity, spending behavior, and repayment patterns, providing a foundation for personalized marketing, risk management, and customer retention strategies.

Chapter 8: Natural Language Processing (NLP) and Sentiment Analysis

Customer reviews contain valuable information about product quality, customer satisfaction, and purchasing behavior. However, analyzing thousands of reviews manually is time-consuming and difficult to scale. This section addresses this challenge by developing Natural Language Processing (NLP) pipeline to automatically classify customer reviews as recommended or not recommended. The solution combines traditional machine learning, deep learning, and transformer-based models to understand customer sentiment and extract actionable business insights.

The goal was not only to predict review sentiment but also to transform unstructured customer feedback into a decision-support tool for improving customer experience and product strategies.

Understanding Customer Feedback Through EDA

The analysis started with 22,641 customer reviews containing 10 features, including review text, ratings, customer information, and product attributes.

Before building predictive models, exploratory data analysis was performed to understand customer behavior and discover hidden patterns.

Key findings included:

- Missing values in review text, titles, and categorical features were identified and handled.
- Customer ratings showed a strong positive skew, indicating that most customers had favorable experiences.
- A strong relationship was observed between higher ratings and product recommendation decisions.
- Review length analysis was performed to determine whether detailed customer feedback was associated with stronger opinions.
- Product categories, departments, and divisions were analyzed to identify areas with higher or lower customer satisfaction.

Business Insight

Although ratings provide a numerical summary of customer satisfaction, customer reviews contain deeper explanations about why customers are satisfied or dissatisfied. Text analysis provides additional intelligence that traditional metrics cannot capture.

Text Processing & Feature Engineering

Customer reviews are naturally unstructured and contain challenges such as informal language, spelling mistakes, and unnecessary words. A complete text processing pipeline was developed to transform raw customer comments into meaningful features.

The preprocessing workflow included:

- Converting text into lowercase format.
- Removing punctuation, special characters, and unnecessary noise.
- Tokenizing sentences into individual words.
- Removing common stopwords.
- Applying lemmatization to convert words into their meaningful base forms.
- Handling missing or incomplete review text.

After cleaning the data, multiple feature extraction techniques were evaluated:

-Traditional Text Representation

- **Bag of Words (BoW):** Captured word frequency patterns.
- **TF-IDF:** Identified words that were most important for distinguishing recommended and non-recommended products.

-Advanced AI-Based Representation

- **Word Embeddings:** Captured numerical representations of words for deep learning models.
- **BERT Embeddings:** Captured contextual meaning by understanding relationships between words within sentences.

Additional customer attributes, including rating, age, and feedback count, were integrated to improve prediction capability.

Modeling Approach

1. Traditional Machine Learning Models

After transforming raw customer reviews into clean and standardized text, the next step was converting textual information into a numerical format that machine learning algorithms could understand. Since machine learning models cannot directly process words, feature extraction techniques such as Bag of Words (BoW) and TF-IDF were applied to represent customer reviews as numerical vectors. These transformed features were then used to develop baseline sentiment classification models using Logistic Regression and Random Forest.

Strength:

Fast, interpretable, and effective for capturing important keywords.

Limitation:

Limited ability to understand word context and complex language patterns.

```
X = df["clean_text"]
y = df["Recommended IND"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# -----
# 1. Bag of Words (BoW)
# -----
bow = CountVectorizer(max_features=5000)
X_train_bow = bow.fit_transform(X_train)
X_test_bow = bow.transform(X_test)

# Logistic Regression (BoW)
lr = LogisticRegression(C=0.01, class_weight = 'balanced', max_iter=1000)
lr.fit(X_train_bow, y_train)
y_pred_lr_bow = lr.predict(X_test_bow)

# Random Forest (BoW)
rf = RandomForestClassifier(n_estimators = 200, max_depth = 4, class_weight="balanced", random_state = 42, n_jobs = -1)
rf.fit(X_train_bow, y_train)
y_pred_rf_bow = rf.predict(X_test_bow)

# -----
# 2. TF-IDF
# -----
tfidf = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

```

# Logistic Regression (TF-IDF)
lr.fit(X_train_tfidf, y_train)
y_pred_lr_tfidf = lr.predict(X_test_tfidf)

# Random Forest (TF-IDF)
rf.fit(X_train_tfidf, y_train)
y_pred_rf_tfidf = rf.predict(X_test_tfidf)

# -----
# Evaluation
# -----

print("LR BoW Accuracy:", accuracy_score(y_test, y_pred_lr_bow))
print("RF BoW Accuracy:", accuracy_score(y_test, y_pred_rf_bow))

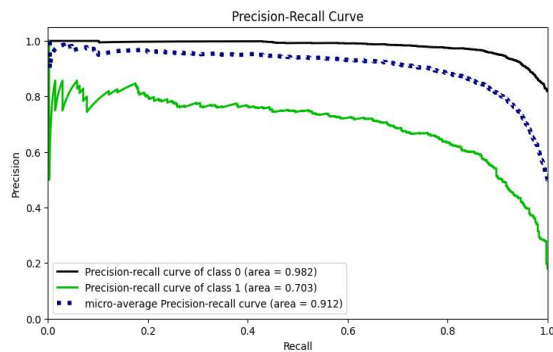
print("LR TF-IDF Accuracy:", accuracy_score(y_test, y_pred_lr_tfidf))
print("RF TF-IDF Accuracy:", accuracy_score(y_test, y_pred_rf_tfidf))

print("\nClassification Report (TF-IDF + LR):")
print(classification_report(y_test, y_pred_lr_tfidf))

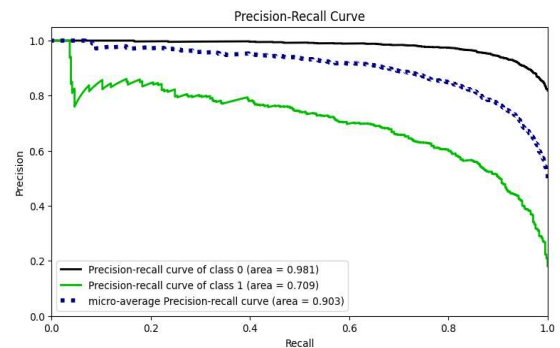
```

Logistic Regression:

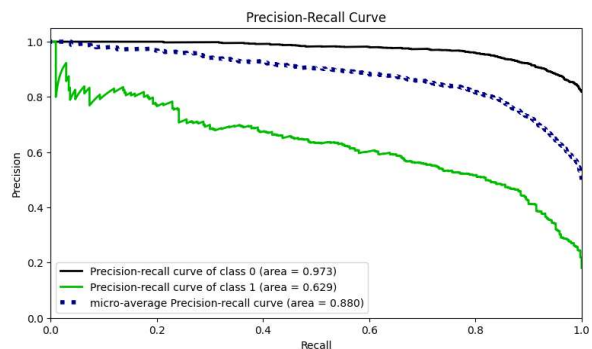
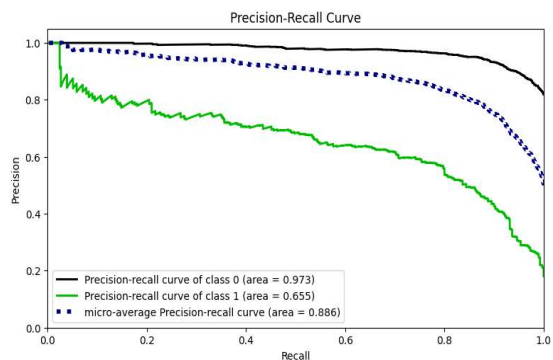
BoW



TF-IDF



Random Forest



2. Deep Learning Approach

A neural network model was developed using text embeddings to capture more complex patterns in customer language.

Techniques applied:

- Tokenize
- Embedding layers
- Dropout regularization
- Neural network optimization

Benefit:

Improved ability to learn nonlinear relationships in text data.

TOKONIZE:

```
num_words = 15001
tokenizer = Tokenizer(num_words=num_words)

tokenizer.fit_on_texts(X_train)

tokenizer.word_index
```

Converting Tokens To Numeric

```
X_train_num_tokens = tokenizer.texts_to_sequences(X_train)
X_test_num_tokens = tokenizer.texts_to_sequences(X_test)

len(X_train_num_tokens)

20376

X_test[:5]
```

Pad_Sequences

```
X_train_pad = pad_sequences(X_train_num_tokens, maxlen=max_tokens, padding="pre")
X_test_pad = pad_sequences(X_test_num_tokens, maxlen=max_tokens, padding="pre")

display(X_train_pad.shape)
display(X_test_pad.shape)

(20376, 104)
(2264, 104)

np.array(X_test_num_tokens[28])

array([ 56, 112, 27, 73, 34, 26, 3, 34, 45, 50, 70, 3, 674,
        2, 38, 323, 493, 3, 24, 4, 90, 404, 16, 33, 71, 8,
        219, 112, 20, 27, 179, 537])
```

Modeling

```
embedding_size = 50

model = Sequential()

model.add(Embedding(input_dim=num_words,
                    output_dim=embedding_size,
                    input_length=max_tokens))

model.add(Dropout(0.2))

model.add(GRU(units=50, return_sequences=True)) # units equal to word embedding - best practice
model.add(Dropout(0.2))

model.add(GRU(units=25, return_sequences=True))
model.add(Dropout(0.2))

model.add(GRU(units=12, return_sequences=False))

model.add(Dense(1, activation='sigmoid'))

optimizer = Adam(learning_rate=0.001)

model.compile(loss='binary_crossentropy',
              optimizer=optimizer,
              metrics=['Recall'])
```

After applying text preprocessing and converting customer reviews into numerical sequences, a Recurrent Neural Network (RNN) based architecture using GRU (Gated Recurrent Unit) layers was developed to capture sequential patterns and contextual information from customer language.

Unlike traditional machine learning models such as Logistic Regression and Random Forest, which rely on manually extracted features such as TF-IDF word frequencies, GRU networks learn meaningful representations directly from text sequences and understand relationships between words across a review.

Model Architecture

The developed GRU model consists of an embedding layer followed by multiple recurrent layers:

- **Embedding Layer**
 - Converts words into dense numerical vectors.
 - Learns semantic relationships between words based on customer review patterns.
 - Vocabulary size: 15,001 words
 - Embedding dimension: 50
- **Three GRU Layers**
 - The first GRU layer captures general sequence patterns from review text.
 - The second GRU layer learns deeper language representations.
 - The final GRU layer compresses the learned information into a 12-dimensional feature representation for classification.
- **Dropout Regularization**
 - Applied between recurrent layers to reduce overfitting and improve model generalization.
- **Dense Output Layer**
 - A sigmoid activation function was used for binary classification:
 - Recommended review
 - Not recommended review

The final architecture contained 772,542 trainable parameters, allowing the model to learn complex patterns from customer language.

Training Performance Analysis

During training, the GRU model showed strong learning capability:

Epoch	Training Recall	Validation Recall
1	77.9%	89.3%
2	90.2%	84.6%
3	92.7%	90.7%
4	95.0%	79.5%
5	95.9%	81.9%

Interpretation

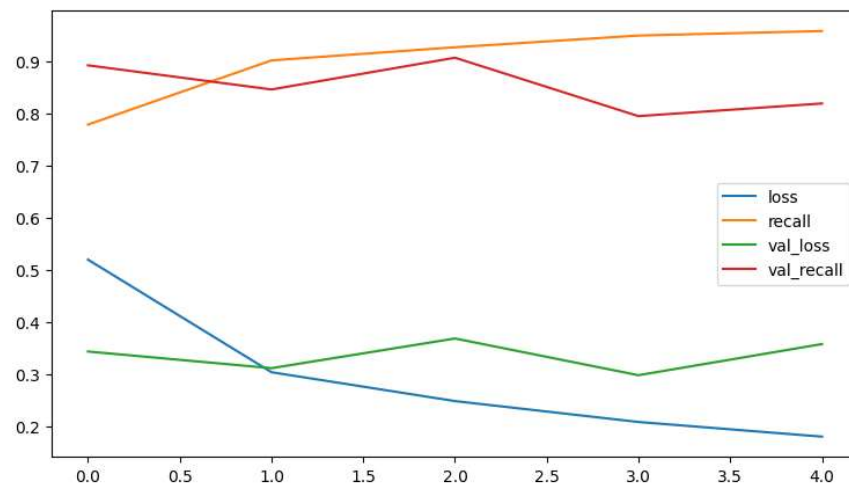
The model quickly learned important linguistic patterns from customer reviews, increasing training recall from 77.9% to 95.9%.

Validation recall remained high (approximately 80–90%), indicating that the model successfully identified most recommended and non-recommended reviews on unseen data. However, the gap between training and validation performance suggests some degree of overfitting due to the complexity of the multi-layer GRU architecture. Additional regularization techniques such as stronger dropout, early stopping, or hyperparameter optimization could further improve generalization.

Conclusion

The GRU deep learning model successfully demonstrated the capability of neural networks to learn complex patterns from customer review text. Compared with traditional TF-IDF-based machine learning approaches, the GRU architecture provides stronger contextual understanding by capturing word relationships and sequential information.

Although the model showed some signs of overfitting due to its high capacity, it achieved strong recall performance and proved that deep learning approaches can effectively transform unstructured customer reviews into actionable customer intelligence.



3. Transformer-Based Model (BERT)

BERT (Bidirectional Encoder Representations from Transformers) is an advanced NLP model that understands the context and meaning of words by analyzing the entire sentence, considering both previous and following words. Unlike traditional models such as TF-IDF, BERT captures deeper semantic relationships and language patterns from customer reviews.

Approach:

- **Pre-training:** BERT learns general language representations from large text datasets using tasks such as Masked Language Modeling.
- **Fine-tuning:** The pretrained model is adapted for sentiment classification by adding a classification layer to predict whether a review is recommended or not.

Table. Challenges & Solutions

Challenge	Solution
Imbalanced positive and negative reviews	Applied evaluation metrics beyond accuracy, including precision, recall, and F1-score
Missing review text	Handled missing values through cleaning and preprocessing strategies
Noisy customer language	Applied text normalization and lemmatization
Overfitting in complex models	Used dropout, regularization, and validation monitoring
High computational cost of BERT	Applied transfer learning and fine-tuning instead of training from scratch

Tokenize

```
from transformers import AutoTokenizer #BertTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# For every sentence...
num_of_sent_tokens = []
for sent in X_train:
    input_ids = tokenizer.encode(sent,
                                add_special_tokens=True)
    num_of_sent_tokens.append(len(input_ids))

print('Max sentence length: ', max(num_of_sent_tokens))

Max sentence length: 162
```

Transformation Vectors to Matrix

```
def transformation(X):
    # set array dimensions
    seq_len = 162

    sentence_tokens = tokenizer(list(X),
                                max_length=seq_len,
                                truncation=True,
                                padding='max_length',
                                add_special_tokens=True)

    return np.array(sentence_tokens['input_ids']), np.array(sentence_tokens['attention_mask'])

Xids_train, Xmask_train = transformation(X_train)
Xids_test, Xmask_test = transformation(X_test)

print("Xids_train.shape :", Xids_train.shape)
print("Xmask_train.shape :", Xmask_train.shape)
print("Xids_test.shape :", Xids_test.shape)
print("Xmask_test.shape :", Xmask_test.shape)
```

```
def create_model():

    model = TFAutoModel.from_pretrained("bert-base-uncased")

    input_ids = Input(shape=(162,), name='input_ids', dtype='int32')
    attention_mask = Input(shape=(162,), name='attention_mask', dtype='int32') # dtype = int32

    embeddings = model.bert(input_ids=input_ids, attention_mask=attention_mask)["pooler_output"] #[1]
    # functional api
    x = Dense(80, activation='relu')(embeddings) #80-160-50.
    x = BatchNormalization()(x)
    x = Dropout(0.1, name="dropout")(x) #0.1
    y = Dense(1, activation='sigmoid', name='outputs')(x)

    return Model(inputs=[input_ids, attention_mask], outputs=y)
```

This project developed an end-to-end NLP pipeline to predict whether customers would recommend a product based on their review text. Multiple approaches were evaluated, ranging from traditional machine learning to deep learning and transformer-based language models.

Models Evaluated

- Logistic Regression (TF-IDF)
- Random Forest (TF-IDF)
- GRU-based Deep Learning
- BERT (Transformer)

Traditional machine learning models established strong baseline performance using TF-IDF features. The GRU model improved contextual understanding by learning sequential relationships between words, while the BERT model achieved the best overall performance by leveraging bidirectional contextual representations, enabling more accurate interpretation of customer sentiment and intent.

Technical Challenges

Several challenges were addressed throughout the NLP pipeline:

- **Noisy text data:** Customer reviews contained slang, misspellings, emojis, punctuation, and inconsistent writing styles. A preprocessing pipeline including text cleaning, tokenization, lemmatization, and stopword removal was applied to standardize the data.
- **High-dimensional features:** TF-IDF generated thousands of features, increasing computational cost and overfitting risk. Feature selection techniques were used to reduce dimensionality, while BERT embeddings provided compact contextual representations.

- **Deep learning overfitting:** The GRU model showed higher training performance than validation performance. Dropout regularization, L2 regularization, and Early Stopping were implemented to improve generalization and reduce overfitting.

Business Impact

The final BERT-based sentiment analysis model enables organizations to automatically analyze thousands of customer reviews, identify dissatisfied customers, detect recurring product issues, and monitor customer satisfaction at scale. By transforming unstructured customer feedback into actionable business intelligence, the solution supports data-driven product improvement, marketing optimization, and customer retention strategies.

Chapter 9: SQL Analytics Layer

A relational database layer implemented with SQLite enabled business querying without ML dependency. The database schema was designed with normalized tables for customers, products, sales, and monthly revenue to support efficient analytical queries.

Business Intelligence Queries

I developed production-ready SQL queries for comprehensive business intelligence. Total revenue and revenue by product category provided insight into product performance and category trends. Top 10 high-value customers by total spend enabled VIP identification and personalized treatment. Revenue trend analysis by month and by shift (morning/evening) revealed seasonal patterns and operational insights. Average order value and customer recency calculations supported cohort analysis and RFM segmentation.

Self-Service Analytics

The SQL analytics layer empowered non-technical stakeholders to access business insights directly. Business users could query key performance indicators including revenue trends, customer behavior metrics, and product performance without engineering support. This democratization of analytics ensured that insights reached decision-makers quickly and reduced dependency on the data engineering team.

Chapter 10: Challenges and Technical Solutions

Class Imbalance in Churn Data emerged as the first major challenge. With only 16% churn cases, models were biased toward predicting the majority class (non-churn). The solution combined SMOTE with Tomek Links and cross-validation to ensure balanced learning and improved generalization. This approach created synthetic samples for the minority class while removing ambiguous boundary cases.

Noisy Unstructured Text Data required careful handling of slang, emojis, misspellings, and domain-specific language. The solution was a multi-stage NLP pipeline using regex for cleaning, spaCy for tokenization, and custom stopword lists to preserve domain-relevant terminology.

Overfitting in Deep Learning Models manifested as training accuracy exceeding 95% while validation lagged at approximately 85%. I implemented Dropout layers (0.2–0.3), L2 regularization to penalize complex weight patterns, and EarlyStopping with patience=3 to prevent excessive training.

High-Dimensional NLP Features created computational strain with TF-IDF generating over 50,000 features. Chi-square feature selection reduced dimensionality to 5,000 features, and BERT embeddings (768-dim) provided a more compact and semantically rich representation.

Feature Redundancy in Behavioral Data required systematic filtering through correlation analysis and permutation importance to identify and remove redundant features, improving model efficiency and interpretability.

Improving Churn Detection

One of the main analytical challenges was customer churn imbalance, where only 16% of customers represented churn cases. Traditional models tended to favor the majority group and failed to identify valuable at-risk customers.

To address this challenge:

- Applied **SMOTE** to improve minority class representation
- Used **Tomek Links** to reduce overlapping data points
- Focused evaluation on **recall and F1-score** rather than accuracy alone

This improved the ability to identify customers requiring retention attention.

Personal Learning: Connecting Analytics with Business Strategy

The most important lesson from this project was that successful analytics is not only about building models , it is about translating data into business actions.

Through this project, I strengthened my ability to:

- Transform raw customer data into meaningful insights
- Connect predictive analytics with business objectives
- Combine SQL, machine learning, NLP, and visualization techniques into a complete analytical workflow
- Deliver solutions that support marketing, customer experience, and strategic decision-making

This project demonstrates my ability to build end-to-end analytics solutions that connect data, customer behavior, and business growth opportunities.

Chapter 12: Future Roadmap

The platform has significant potential for future enhancement and expansion. Real-time prediction via FastAPI or Flask would enable on-demand churn risk assessment for individual customers. Multi-modal learning could integrate text, transaction, and customer support data for richer customer understanding. Explainable AI using SHAP and LIME would provide interpretable explanations for model predictions, building trust with business users. A/B testing integration would enable systematic experimentation to validate and optimize marketing strategies. An automated retraining pipeline would ensure models remain accurate as customer behavior evolves over time.

Chapter 13: Conclusion

This project demonstrates a complete Customer Intelligence Platform that integrates data engineering, machine learning, deep learning, NLP, SQL analytics, and business intelligence into a unified system. Rather than producing static dashboards that describe historical performance, the system enables predictive and prescriptive decision-making, helping businesses retain valuable customers before they leave, understand customer sentiment at scale, optimize marketing strategies

through behavioral segmentation, improve product performance through feedback analysis, and make proactive data-driven decisions.

The platform successfully shifts organizations from reactive reporting to proactive, data-driven customer strategy optimization. By combining advanced machine learning techniques with practical business application, this system transforms from a technical ML project into a real-world business intelligence ecosystem that delivers measurable value.

Appendix : Technology Stack

Data Processing and Analysis: Pandas, NumPy, Scikit-learn for statistical analysis and feature engineering.

Natural Language Processing: NLTK and spaCy for text preprocessing, tokenization, and lemmatization.

Machine Learning: XGBoost and LightGBM for gradient boosting models optimized for performance.

Deep Learning: TensorFlow and Keras for neural network implementation, Hugging Face for BERT transformer models.

Database and Analytics: SQLite with python for relational database management.

Visualization: Plotly for interactive visualizations, Matplotlib for static charts and publication-quality figures.

Model Explainability: SHAP for model interpretability and feature importance analysis.